

17 Day 11: 异步编程基础（下）——fetch API 与 async/await

最终交付成果

成果

验证方式

用 `fetch()` 请求一个公开 API，拿到真实数据

控制台打印出从网络请求回来的真实 JSON 数据

理解 `async/await` 是 Promise 的"语法糖"

能把 Step 5（Day 10）写的 Promise 代码改写成 `async/await` 版本

用 `try/catch` 处理异步请求中的错误

故意写错请求地址，能看到 `catch` 捕获到错误并友好提示

把真实数据渲染到作品集页面（选做加分项）

页面上出现一段由网络数据动态生成的内容

完成一次 `git pull` + `push` 的完整协作流程

本地能成功拉取远程最新代码，再推送本次改动

最终文件结构

text

my-portfolio/

├─ index.html

├─ about.html

├─ styles.css

└─ script.js

← 今天新增 fetch/async-await 练习代码

📅 Step 架构总览

Step	内容	预计用时	目标
Step 1	晨会：回顾 + 目标 + 概念预热	10min	理解 fetch 和 async/await 要解决什么问题
Step 2	<code>fetch()</code> 基础用法	15min	用 fetch 请求真实数据并拿到 JSON
Step 3	<code>async/await</code> 语法	15min	用更简洁的写法改写 Promise 链
Step 4	<code>try/catch</code> 错误处理	15min	优雅处理请求失败的情况
Step 5	综合练习：渲染真实数据到页面（选做）	15min	把 fetch 到的数据用 Day 8 学的 map 技巧渲染出来
Step 6	Git 协作练习： <code>pull</code> + <code>push</code>	10min	掌握拉取远程最新代码的流程
Step 7	Code Review + 自评	10min	检验理解

🔥 Step 1：晨会

昨日回顾（2分钟）

你昨天完成了回调函数、回调地狱问题、Promise 基础语法（`resolve`/`reject`/`.then()`/`.catch()`），并完成了一次日常的 `git push`。

快速自测：

- 1. `.then()` 和 `.catch()` 分别对应 Promise 的哪种结果？
- 2. 回调地狱最大的问题是什么？

今日目标说明（3分钟）

目标	完成标志
掌握 <code>fetch()</code> 请求数据	能从一个公开 API 拿到真实 JSON 数据
掌握 <code>async/await</code> 语法	能把 Promise 链改写成更易读的 <code>async/await</code> 形式
掌握异步错误处理	能用 <code>try/catch</code> 优雅处理请求失败
理解 Git 协作的基本流程	能说出为什么推送前要先 <code>pull</code>

概念预热 (3分钟)

问题： Day 10 写的 Promise 代码里，你有没有觉得 `.then().then().catch()` 这种链式写法虽然比回调地狱好，但读起来还是有点"不像平常写代码的感觉"（不是从上到下一行行的语句）？如果能把异步代码写得**看起来跟同步代码一样**，是不是会更直观？

PM 视角导入 (2分钟)

text

Promise (Day 10)：解决了"回调地狱"的嵌套问题，但读起来还是"链式调用"的味道
async/await (今天)：让异步代码"看起来"和同步代码一模一样，可读性再上一个台阶

现实场景：

- `fetch()` 是浏览器内置的"发起网络请求"的标准方式，公司项目里请求后端接口几乎都靠它
- `async/await` 已经是目前业界写异步代码的主流方式，比纯 Promise 链更常见

学会这两个，你就具备了"从服务器拿真实数据"的能力——
这是让作品集从"静态网页"变成"动态应用"的关键一步。

另外，今天要第一次练习 `git pull`——
真实协作场景里，别人可能已经把代码推送到远程，
你必须先拉取最新版本，再推送自己的改动，否则可能产生冲突。

课前预习：

1. 想好两个自测题的答案
2. 想好"async/await 让代码更像同步"这句话的直觉理解
3. 准备好了，回复"进入 Step 2"